Snowflake Practice Assessments – MS2

4 Easy + 3 Medium | Screenshot-Based Practice Pack

Easy: 10 Marks / 20 Minutes • Medium: 20 Marks / 45 Minutes

Design note: Each assessment uses a different business domain and is intentionally scoped to fit the stated target duration. Tasks are outcome-focused and screenshot evidence maps directly to the rubric.

Easy 1 – Snowflake Time Travel, Data Load/Unload & Data Sharing

Difficulty	Easy	Total Marks	10
Target Duration	20 minutes	Dataset	E1_Logistics_Shipments.csv

Curriculum Coverage

Snowflake named stages, COPY INTO data loading, Time Travel, data unloading, secure views, secure data sharing.

Problem Description

You are working as a Snowflake developer for a logistics company. Shipment data must be loaded into Snowflake, a recent accidental change must be investigated using Time Travel, delivered shipments must be unloaded for downstream use, and a non-sensitive shipment view must be prepared for secure sharing.

Dataset

- Dataset: E1_Logistics_Shipments.csv
- Domain: Logistics & Shipment Operations
- Columns: ShipmentID, ShipmentDate, CustomerID, OriginCity, DestinationCity, Region, TransportMode, ShipmentStatus, WeightKg, FreightAmount

Assessment Requirements

1. Create a Snowflake table for the dataset, create/use a named internal stage, and load the CSV using COPY INTO. Validate the final row count.

2. Perform exactly one UPDATE on FreightAmount for a selected shipment and one DELETE on a different shipment. Use Time Travel to display the affected records as they existed before the changes.

3. Unload only Delivered shipments to a stage in CSV format and verify the unloaded file(s) using LIST.

4. Create a secure view exposing ShipmentID, ShipmentDate, OriginCity, DestinationCity, Region, TransportMode, ShipmentStatus and FreightAmount while excluding CustomerID.

5. Create a secure share for the secure view, grant the required database/schema/view privileges, and validate the share using SHOW SHARES.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that SQL/code and the corresponding result are visible together.

1. Load validation: table row count and sample rows.

2. Time Travel output showing original values before UPDATE/DELETE.

3. Unload command plus LIST output showing generated file(s).

4. Secure view definition/output.

5. Share creation/grants plus SHOW SHARES output.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Dataset is loaded correctly and row count is validated.	2
Task 2	Time Travel correctly retrieves the	2

	historical state of the modified/deleted records.	
Task 3	Delivered shipments are unloaded successfully and staged files are visible.	2
Task 4	Secure view correctly excludes CustomerID and returns required columns.	2
Task 5	Secure share is created with correct object access and visible in SHOW SHARES.	2
Total		**10**

Easy 2 – JSON Semi-Structured Data, Accessing JSON & LATERAL FLATTEN

Difficulty	Easy	**Total Marks**	10
Target Duration	20 minutes	**Dataset**	E2_Insurance_Claims.json

Curriculum Coverage

VARIANT, JSON path notation, nested object access, arrays, LATERAL FLATTEN, structured analytical views.

Problem Description

An insurance platform stores claims as JSON documents. You must load the raw documents, access nested policy and customer attributes, flatten claim-item arrays, and produce a structured analytical result.

Dataset

- Dataset: E2_Insurance_Claims.json (newline-delimited JSON)
- Domain: Insurance Claims
- Top-level attributes: claim_id, claim_date, claim_type, status
- Nested objects: policy, customer
- Array: items (item_code, description, amount)

Assessment Requirements

1. Create a table with a VARIANT column, create/use an appropriate JSON file format and stage, and load the JSON file. Display sample raw documents.

2. Access nested JSON attributes and display Claim ID, Claim Date, Policy ID, Plan, Customer ID, Customer Name, City and Status using JSON path expressions.

3. Use LATERAL FLATTEN on the items array and display Claim ID, Item Code, Description and Amount as individual rows.

4. Create a view named VW_CLAIM_ITEMS that combines claim-level and item-level details and includes ITEM_AMOUNT as a numeric value.

5. Using the view, show total claim-item amount and item count grouped by Claim Type, sorted by total amount descending.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that SQL/code and the corresponding result are visible together.

1. Raw VARIANT data successfully loaded.

2. Nested JSON attributes extracted correctly.

3. LATERAL FLATTEN output with one row per claim item.

4. VW_CLAIM_ITEMS output.

5. Claim-type aggregation output.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	JSON is loaded into VARIANT correctly.	2

Task 2	Nested JSON attributes are accessed correctly.	2
Task 3	LATERAL FLATTEN expands the items array correctly.	2
Task 4	Structured view contains the required claim/item fields.	2
Task 5	Aggregation produces correct claim-type totals and counts.	2
Total		**10**

Easy 3 – Cortex Analyst

Difficulty	Easy	Total Marks	10
Target Duration	20 minutes	Dataset	E3_Telecom_Usage.csv

Curriculum Coverage

Cortex Analyst, semantic model/semantic view, dimensions, measures, natural-language questions, generated SQL, result validation.

Problem Description

A telecom operations team wants business users to ask natural-language questions about usage and revenue. Configure a compact semantic layer on a single usage table, ask business questions in Cortex Analyst, and validate one answer independently using SQL.

Dataset

- Dataset: E3_Telecom_Usage.csv
- Domain: Telecom Usage & Billing
- Columns: UsageID, UsageDate, CustomerSegment, PlanName, Region, DataGB, VoiceMinutes, SMSCount, ChargeAmount

Assessment Requirements

1. Load the telecom dataset into a Snowflake table and validate the row count plus sample records.

2. Create/configure a semantic model or semantic view for the table with dimensions CustomerSegment, PlanName, Region and UsageDate, and measures Total Charge, Total Data GB and Average Charge.

3. Ask Cortex Analyst: “What is the total charge amount by region?” and display the response.

4. Ask Cortex Analyst: “Which plan has the highest average data usage?” and display the generated SQL together with the result.

5. Run an independent SQL query for total ChargeAmount by Region and verify that the result agrees with Cortex Analyst.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that SQL/code and the corresponding result are visible together.

1. Dataset validation output.

2. Semantic model/view showing required dimensions and measures.

3. First Cortex Analyst response.

4. Second Cortex Analyst response with generated SQL.

5. Independent SQL validation result.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Dataset is loaded and validated.	2
Task 2	Semantic configuration contains the required business dimensions/measures.	2

Task 3	Cortex Analyst correctly answers regional total charge.	2
Task 4	Cortex Analyst returns a valid plan-level answer and generated SQL.	2
Task 5	Independent SQL confirms the Cortex Analyst regional result.	2
Total		**10**

Easy 4 – RBAC to Row Access Policy (RAP)

Difficulty	Easy	Total Marks	10
Target Duration	20 minutes	Dataset	E4_Healthcare_Appointments.csv

Curriculum Coverage

RBAC, role creation, privilege grants, role-to-region mapping, Row Access Policy, policy application and validation.

Problem Description

A hospital network requires regional operations roles to see only appointments belonging to their assigned region, while an administrator must see all regions. Implement the requirement using Snowflake RBAC and a Row Access Policy.

Dataset

- Dataset: E4_Healthcare_Appointments.csv
- Domain: Healthcare Operations
- Columns: AppointmentID, AppointmentDate, PatientID, PatientName, Region, Hospital, Department, DoctorName, ConsultationFee, AppointmentStatus

Assessment Requirements

1. Load the dataset into a Snowflake table and validate that multiple regions are present.

2. Create roles HR_NORTH, HR_SOUTH and HR_ADMIN. Grant only the privileges required to query the assessment objects.

3. Create a role-to-region mapping table and create a Row Access Policy that allows regional roles to see only their mapped region while HR_ADMIN can see all rows. Apply the policy to Region.

4. Switch to HR_NORTH and validate that only NORTH rows are returned. Then switch to HR_SOUTH and validate that only SOUTH rows are returned.

5. Switch to HR_ADMIN and validate that records from all available regions are visible.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that SQL/code and the corresponding result are visible together.

1. Source table validation showing multiple regions.

2. Role creation and grants.

3. Mapping table plus RAP definition/application.

4. Regional-role validation outputs.

5. HR_ADMIN output showing all regions.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Healthcare data is loaded and multi-region data is confirmed.	2
Task 2	Required roles and query privileges are created correctly.	2
Task 3	Role mapping and RAP logic are correctly created/applied.	2

Task 4	Regional roles can see only their assigned rows.	2
Task 5	HR_ADMIN can see rows across all regions.	2
Total		**10**

Medium 1 – Snowpipe Auto-Ingest with Incremental Validation

Difficulty	Medium	Total Marks	20
Target Duration	45 minutes	Dataset	M1_Snowpipe_Batches.zip (3 CSV files)

Curriculum Coverage

Snowpipe, external stages, AUTO_INGEST, continuous/incremental loading, SYSTEM$PIPE_STATUS, COPY_HISTORY/load history, validation.

Problem Description

A warehouse IoT platform produces sensor files continuously. Configure Snowpipe to ingest each arriving batch into Snowflake automatically and prove that the pipeline processes incremental files without duplicating or losing earlier data.

Dataset

- Dataset package: M1_Snowpipe_Batches.zip
- Files: warehouse_sensor_batch_01.csv, warehouse_sensor_batch_02.csv, warehouse_sensor_batch_03.csv
- Domain: Warehouse IoT Monitoring
- Columns: EventID, EventTimestamp, WarehouseID, Zone, SensorType, ReadingValue, ReadingUnit, AlertLevel
- Prerequisite for timed practice: trainer provides an accessible S3 location/storage integration and event-notification setup.

Assessment Requirements

1. Create the target SENSOR_EVENTS table, CSV file format and external stage for the provided cloud location. Confirm that batch 01 is visible in the stage.

2. Create a Snowpipe with AUTO_INGEST = TRUE that loads staged sensor files into SENSOR_EVENTS. Display the pipe definition/status.

3. Place batch 01 in the source location and validate the initial load using target row count and sample records.

4. Place batches 02 and 03 at different times. Validate incremental ingestion by recording the row count after each arrival and confirming all EventID values remain unique.

5. Review pipe status and load history. Then show event counts by AlertLevel and confirm that no file from the three provided batches failed to load.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that SQL/code and the corresponding result are visible together.

1. Stage/table/file-format validation.

2. Snowpipe creation plus pipe status/definition.

3. Initial batch load result and row count.

4. Incremental row-count evidence after batches 02 and 03.

5. Load history/status plus AlertLevel summary.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Target, stage and file format are configured correctly.	4
Task 2	Snowpipe is correctly created for automated ingestion.	4
Task 3	Initial batch loads successfully with correct records.	4
Task 4	Incremental batches load without loss or duplicate EventID values.	4
Task 5	Pipe status/load history and final business validation confirm successful ingestion.	4
Total		**20**

Medium 2 – Snowpark Transaction Processing

Difficulty	Medium	Total Marks	20
Target Duration	45 minutes	Dataset	M2_Banking_Transactions.csv

Curriculum Coverage

Snowpark Python DataFrames, schema inspection, data cleaning, expressions, filtering, aggregations, save_as_table, analytical validation.

Problem Description

A bank needs a Snowpark Python pipeline to clean transaction data and produce branch-level insights. The source intentionally contains duplicate rows and null transaction amounts so that data-quality handling can be demonstrated.

Dataset

- Dataset: M2_Banking_Transactions.csv
- Domain: Banking Transactions
- Columns: TransactionID, TransactionDate, CustomerID, BranchID, Region, TransactionType, Channel, TransactionAmount, TransactionStatus
- Data quality: 3 duplicate rows and 3 records with null TransactionAmount.

Assessment Requirements

1. Using Snowpark Python, read the BANK_TRANSACTIONS table into a DataFrame and display schema, source row count and sample records.

2. Clean the DataFrame by removing duplicate TransactionID records, excluding null TransactionAmount rows, and retaining only TransactionStatus = SUCCESS. Display source count, clean count and removed count.

3. Create VALUE_CATEGORY using Snowpark expressions: LOW < 10,000; MEDIUM 10,000–49,999.99; HIGH >= 50,000. Display a representative sample.

4. Aggregate the clean DataFrame by BranchID to calculate transaction count, total amount, average amount and maximum amount. Sort by total amount descending and save the result as BRANCH_TRANSACTION_SUMMARY.

5. Filter HIGH transactions and aggregate by Region and Channel to produce HIGH_VALUE_COUNT and HIGH_VALUE_TOTAL. Display the top results ordered by total amount descending.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that SQL/code and the corresponding result are visible together.

1. Snowpark schema/count/sample output.

2. Cleaning validation with source, clean and removed counts.

3. VALUE_CATEGORY output.

4. Branch summary output plus persisted table validation.

5. High-value Region/Channel analysis.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Snowpark DataFrame correctly	4

	accesses and inspects the source.	
Task 2	Duplicate/null/status cleaning is correctly implemented and validated.	4
Task 3	VALUE_CATEGORY logic is correct.	4
Task 4	Branch aggregation metrics are correct and persisted successfully.	4
Task 5	High-value transaction analysis is correctly filtered, grouped and ordered.	4
Total		**20**

Medium 3 – Streamlit in Snowflake Interactive Dashboard

Difficulty	Medium	Total Marks	20
Target Duration	45 minutes	Dataset	M3_Hotel_Bookings.csv

Curriculum Coverage

Streamlit in Snowflake, Snowflake data access, KPI metrics, filters, charts, interactive tables, business analysis.

Problem Description

A hotel group wants a compact Streamlit dashboard for booking performance. Build an interactive application that reads Snowflake data, responds to filters, and summarizes booking and revenue metrics within a single page.

Dataset

- Dataset: M3_Hotel_Bookings.csv
- Domain: Hospitality & Hotel Operations
- Columns: BookingID, BookingDate, CheckInDate, HotelName, City, RoomType, GuestSegment, BookingChannel, Nights, Guests, BookingStatus, RoomRevenue

Assessment Requirements

1. Load the dataset into a Snowflake table and create a Streamlit in Snowflake app titled "Hotel Booking Performance Dashboard". Connect the app to the table using the active Snowpark session.

2. Display three KPI metrics that update from the data: Total Bookings, Confirmed/Checked-Out Revenue, and Average Nights.

3. Add two interactive filters: City and BookingStatus. Ensure selections change the KPI values and downstream displays.

4. Create two different visualizations: Room Revenue by Hotel and Monthly Booking Trend. Use appropriate chart types and readable labels.

5. Add a filtered detail table and a Top 5 Hotels by Room Revenue summary. Demonstrate that both respond to the selected filters.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that SQL/code and the corresponding result are visible together.

1. Running app with title and connected data.

2. Three KPI metrics.

3. Filter controls with visibly changed results.

4. Two required visualizations.

5. Filtered detail table plus Top 5 Hotels summary.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Streamlit app runs and reads Snowflake data successfully.	4
Task 2	All required KPIs are correctly calculated from data.	4

Task 3	City and BookingStatus filters function correctly.	4
Task 4	Both required business visualizations are correct and readable.	4
Task 5	Detail table and Top 5 summary are correct and filter-responsive.	4
Total		**20**

Practice Pack Summary

Assessment	Domain	Topic	Marks	Target Time
Easy 1	Logistics	Time Travel / Load / Unload / Sharing	10	20 min
Easy 2	Insurance	JSON / LATERAL FLATTEN	10	20 min
Easy 3	Telecom	Cortex Analyst	10	20 min
Easy 4	Healthcare	RBAC / RAP	10	20 min
Medium 1	IoT / Warehousing	Snowpipe	20	45 min
Medium 2	Banking	Snowpark	20	45 min
Medium 3	Hospitality	Streamlit	20	45 min

Exam-plan fit: selecting any 2 Easy + any 2 Medium gives 60 marks. At 20 + 20 + 45 + 45 minutes, the implementation target is 130 minutes, leaving approximately 5 minutes within a 135-minute window for final screenshot review and submission.